PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

2. Вступление-обращение

Содержание

- 3. Профессиональный кейс
- 4. Требования к версии 2.0
- 5. Краткий теоретический минимум
- 6. Подготовка программной среды и данных
- 7. Исходная версия прототипа 1.0
- 8. Основные практические задачи
- 9. Дополнительные задания
- 10. Критерии оценивания
- 11. Чек-лист самопроверки
- 12. Итог занятия и комплект сдачи

Здравствуйте! На предыдущем этапе проекта был собран первый прототип классификатора заявок. Он способен присвоить тексту одну из категорий, но профессиональное решение оценивают не только по факту запуска. Пользователю важно, какие данные принимает система, что она делает с ошибочными значениями, насколько понятен результат и что произойдет при неуверенном прогнозе. Сегодня Вы превратите демонстрационную версию в более управляемый прототип: уточните входные поля, свяжете каждое изменение с требованием, добавите проверки и проведете приемочные тесты. К концу занятия у Вас будет версия 2.0, для которой можно объяснить не только код, но и границы применимости. Работающий код еще не означает, что система готова к использованию. Например, программа может правильно классифицировать обычную заявку, но завершиться с ошибкой при пустом поле или выдать уверенный ответ для слишком общего текста. Поэтому надежность прототипа будет оцениваться как последовательность этапов: проверка входа, получение прогноза, оценка уверенности, расчет приоритета и формирование результата.

3. Профессиональный кейс

Учебный центр использует внутреннюю службу технической поддержки. Заявки поступают в свободной форме, поэтому сотруднику приходится вручную определять категорию проблемы и приоритет. Первый прототип «ServiceDesk Assistant 1.0» классифицирует текст как «сеть», «доступ», «ПО» или «оборудование». После демонстрации заказчик сформулировал дополнительные требования: система должна проверять поля, показывать уверенность, не принимать решение при недостаточных данных, вычислять приоритет по прозрачным правилам и возвращать единый структурированный результат.
Категория и приоритет отвечают на разные вопросы. Категория показывает, какому направлению поддержки относится проблема, а приоритет — насколько срочно ее следует рассматривать. Например, неисправность принтера относится к оборудованию, но ее приоритет зависит от числа затронутых пользователей, длительности недоступности и наличия временного решения.

Главная идея занятия
Доработка считается обоснованной, если для каждого изменения можно указать: какое требование выполняется, какую проблему устраняет изменение и каким тестом подтвержден результат.
Такая связь образует проверяемую цепочку «требование → реализация → тест». Если хотя бы одно звено отсутствует, нельзя однозначно доказать, что изменение действительно решает поставленную задачу.

4. Требования к версии 2.0

Идентификаторы R1–R7 нужны для трассировки: на них можно ссылаться в коде, журнале проблем и отчете о приемке. Критерий приемки переводит общее пожелание в наблюдаемый результат, который можно проверить на конкретных данных.

ID	Область	Требование	Критерий приемки
R1	Входные поля	ticket_id, text, users_affected, outage_minutes, has_workaround	Все пять полей обработаны; отсутствие обязательного поля отражается в errors
R2	Проверка данных	Текст 5–500 символов; users 1–500; outage 0–1440; ответ о временном решении нормализуется	Некорректный объект не передается модели и правилам приоритета
R3	Категория	Один из классов: сеть, доступ, ПО, оборудование	Accuracy на фиксированной проверочной выборке не ниже 0,80

ID	Область	Требование	Критерий приемки
R4	Неуверенный прогноз	Если максимальная вероятность ниже 0,60, результат передается специалисту	status = manual_review, причина отражена в explanation
R5	Приоритет	Низкий, средний, высокий или критический по users, outage и workaround	Все обычные и граничные тесты правил пройдены
R6	Формат результата	Словарь с полями status, ticket_id, category, confidence, priority, explanation, errors	Пакетная обработка возвращает запись для каждой строки
R7	Ограничения	В тексте не используются ФИО, телефоны, пароли и другие персональные данные	В отчете зафиксированы ограничения и случаи ручной проверки

Обратите внимание: выполнение R3 не заменяет R2 и R4. Даже модель с достаточной accuracy не должна получать заведомо некорректные данные или автоматически принимать решение при уверенности ниже установленного порога.

5. Краткий теоретический минимум

5.1. Требование, реализация и проверка

Требование описывает ожидаемое поведение системы с точки зрения пользователя. Реализация — конкретный код, правило или настройка, с помощью которых это поведение создается. Проверка — заранее подготовленный пример, позволяющий однозначно увидеть, выполнено ли требование. Формулировка «система должна работать хорошо» непригодна для приемки, а формулировка «все строки с users_affected = 0 отклоняются до классификации» проверяема.

Один тест может подтверждать только часть требования. Например, проверка users_affected = 0 показывает отклонение значения ниже диапазона, но не подтверждает корректную обработку верхней границы 500 или строки, которую невозможно преобразовать в целое число. Поэтому для одного требования обычно требуется несколько тестов.

5.2. Признаки и входные данные

Для модели классификации главным признаком является текст заявки. Версия 2.0 использует два представления текста: словесные признаки помогают распознавать целые слова и устойчивые сочетания, а символьные фрагменты частично учитывают формы слов и опечатки. Числовые поля users_affected и outage_minutes не используются для определения категории: они описывают масштаб события и нужны правилу расчета приоритета. Такое разделение делает логику понятнее и уменьшает риск ложных зависимостей.

Например, биграмма «не работает» сохраняет информацию о соседстве двух слов, а символьные фрагменты позволяют обнаружить частичное сходство между близкими написаниями. При этом символьные признаки не гарантируют правильную обработку любой опечатки: они лишь дают модели дополнительную информацию.

Частая ошибка — передавать числовые поля в классификатор категории только потому, что они уже присутствуют в заявке. В данном прототипе это нарушило бы принятую архитектуру: категория определяется по содержанию проблемы, а масштаб влияет на приоритет.

5.3. Уверенность и ручная проверка

Вероятность класса не является гарантией правильности, но помогает обнаруживать неоднозначные тексты. Если заявка «что-то не работает» не содержит признаков конкретной проблемы, автоматическое решение опасно. Поэтому прототип сравнивает максимальную вероятность с порогом 0,60. Ниже порога он не скрывает сомнение, а устанавливает status = manual_review.

Сравнивается именно наибольшая из вероятностей всех классов. Значение confidence = 0,60 проходит условие принятия, поскольку ручная проверка требуется только при confidence < 0,60. Порог является правилом прототипа, а не доказательством того, что все прогнозы выше него верны.

При status = manual_review категорию и рассчитанную уверенность можно сохранить для специалиста как вспомогательную информацию, но их нельзя представлять как окончательное автоматическое решение.

5.4. Граничные и негативные тесты

Обычный тест проверяет типичную рабочую ситуацию. Граничный тест использует значение непосредственно перед порогом, на пороге и сразу после него. Негативный тест содержит недопустимые данные: пустой текст, отрицательное время, неизвестный ответ или пропущенное поле. Профессиональная проверка включает все три группы.

Например, для порога 20 пользователей полезны значения 19 и 20: первое должно остаться в диапазоне 5–19, а второе — перейти в диапазон 20–49. Проверка только значения 20 не обнаружит ошибку, при которой условие случайно записано как users_affected > 20.

6. Подготовка программной среды и данных

Работу можно выполнять в Jupyter Notebook, Google Colab или локальной среде Python. Для локальной среды при необходимости установите библиотеки:

```
pip install pandas scikit-learn
```

После установки рекомендуется выполнять ячейки по порядку: сначала импортировать библиотеки и сформировать данные, затем разделить выборку и обучить модели. Иначе переменные из предыдущих этапов могут отсутствовать или содержать результаты старого запуска.
Скопируйте код формирования учебного набора. В нем по 16 примеров каждого класса, всего 64 записи. Данные синтетические и не содержат персональной информации.

```
import pandas as pd

TEXTS_BY_CATEGORY = {
"сеть": [
"не работает интернет в кабинете", "пропал wi fi на этаже",
"нет подключения к сети", "роутер не раздает интернет",
"очень медленный интернет", "компьютер не видит локальную сеть",
"обрыв сетевого соединения", "не открываются сайты через сеть",
"точка доступа отключилась", "кабель сети не подключен",
"интернет часто пропадает", "нет связи с сервером по сети",
"wi fi подключен без доступа в интернет", "сетевой адаптер отключен",
"пинг до сервера очень высокий", "не удается получить ip адрес"
],
"доступ": [
"не могу войти в учетную запись", "забыл пароль от портала",
"аккаунт заблокирован после попыток входа", "не приходит код подтверждения",
"нет прав доступа к папке", "система пишет неверный логин",
"нужно восстановить пароль", "не открывается личный кабинет",
"пользователь не авторизуется", "доступ запрещен для сотрудника",
"истек срок действия пароля", "не могу пройти двухфакторную авторизацию",
"нужно выдать права на каталог", "логин не найден в системе",
"учетная запись отключена", "не получается сменить пароль"
],
"по": [
"программа не запускается", "приложение выдает ошибку",
"не устанавливается обновление программы", "редактор зависает при запуске",
"ошибка лицензии в приложении", "программа закрывается сама",
"не работает функция экспорта", "после обновления приложение тормозит",
"не открывается файл в программе", "сервис показывает программную ошибку",
"приложение зависает при сохранении", "не устанавливается драйвер программы",
"ошибка при запуске модуля", "программа показывает белое окно",
"не работает кнопка в приложении", "после обновления появились сбои"
],
"оборудование": [
"принтер не печатает", "сломалась клавиатура",
"монитор не включается", "компьютер сильно шумит",
"мышь не реагирует", "сканер зажевывает бумагу",
"проектор не показывает изображение", "ноутбук не заряжается",
"на системном блоке не горит индикатор", "принтер печатает полосами",
"веб камера не включается", "наушники не воспроизводят звук",
```

```
"перегревается ноутбук", "диск издает странный звук",
"клавиша на клавиатуре западает", "монитор мерцает"
]
}

rows = []
for category, texts in TEXTS_BY_CATEGORY.items():
for index, text in enumerate(texts):
rows.append({"text": text, "category": category})

df = pd.DataFrame(rows)
print(df.shape)
print(df["category"].value_counts())
```

Ожидаемый размер набора: 64 строки и 2 столбца. В каждом классе должно быть 16 примеров.
Проверка размера и распределения классов нужна до обучения. Если одна категория случайно потеряна или число строк изменилось, результаты следующих этапов уже нельзя корректно сравнивать с ожидаемыми.

7. Исходная версия прототипа 1.0

Версия 1.0 запускается, но не выполняет часть требований. Сначала изучите код, не исправляя его.
При анализе отделяйте ошибки, которые мешают выполнению программы, от логических ошибок, при которых программа запускается, но возвращает неверный результат. Логическая ошибка особенно опасна: внешне функция работает, однако отдельные ветви могут быть недостижимы.

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
df["text"], df["category"],
test_size=0.25,
random_state=4,
stratify=df["category"]
)

model_v1 = Pipeline([
("tfidf", TfidfVectorizer()),
("classifier", LinearSVC())
])
model_v1.fit(X_train, y_train)
print("accuracy_v1 =", accuracy_score(y_test, model_v1.predict(X_test)))

def priority_v1(users_affected, outage_minutes, has_workaround):
score = 0
if users_affected >= 20:
score += 3
elif users_affected >= 5:
score += 1

if outage_minutes >= 60:
score += 3
elif outage_minutes >= 15:
score += 1

if has_workaround == "нет":
score += 2

if score >= 5:
return "высокий", score
elif score >= 8:
return "критический", score
elif score >= 2:
```

```
    return "средний", score
    return "низкий", score

def process_v1(ticket):
    category = model_v1.predict([ticket["text"]])[0]
    priority, score = priority_v1(
        int(ticket["users_affected"]),
        int(ticket["outage_minutes"]),
        ticket["has_workaround"].lower()
    )
    return {"category": category, "priority": priority, "score": score}
```

Проследите порядок условий в priority_v1. Значение score, равное 8 или больше, сначала удовлетворяет условию score >= 5, поэтому выполнение завершается возвратом высокого приоритета. До следующей ветви elif управление уже не доходит, и критический класс становится недостижимым.
Кроме того, прямые обращения по ключам и вызовы int(...) без предварительной проверки могут вызвать исключение. Такой объект не превращается в контролируемый результат с errors, а прерывает обработку.

8. Основные практические задачи

Выполняйте задачи последовательно. Задача №1 является входной и максимально простой. Каждая следующая задача добавляет новый слой надежности.
После каждого этапа сохраняйте фактический вывод программы. Это позволит отличить ожидаемый результат от реально полученного и упростит поиск этапа, на котором возникло расхождение.

9. Дополнительные задания

Дополнительные задания выполняются после пяти основных и не снижают оценку при отсутствии решения.

10. Критерии оценивания

Элемент	Критерий	Баллы
Задача №1	Требования сопоставлены с недостатками; правильно объяснена недостижимость критического класса	3
Задача №2	Валидация и нормализация выполняются без аварийного завершения; B01–B04 обработаны корректно	4
Задача №3	Версия 2.0 обучена; рассчитаны accuracy и вероятности; сделан обоснованный вывод	4
Задача №4	Гибридная обработка соответствует R4–R6; границы приоритета реализованы верно	5
Задача №5	Выполнена приемка, заполнен журнал, зафиксированы ограничения и итоговый вывод	4

Максимум — 20 баллов. Рекомендуемый зачет — 14 баллов. Критическая ошибка: некорректные входные данные передаются модели или пользователю возвращается автоматическое решение при confidence < 0,60.
Критическая ошибка относится к последовательности обработки, а не только к качеству классификации. Даже правильная категория не засчитывается как безопасный результат, если она была автоматически возвращена при нарушенном пороге или после неуспешной валидации.

11. Чек-лист самопроверки

☐ Я сохранил исходную версию 1.0 отдельно от доработанной версии.
☐ Моя функция prepare_ticket не завершается исключением на B01–B04.
☐ Я использую фиксированное разделение random_state=4 и stratify.
☐ Я рассчитал accuracy обеих версий на одной и той же выборке.
☐ При confidence < 0,60 выставляется manual_review.
☐ Границы 19/20 и 49/50, 59/60 и 239/240 проверены отдельными тестами.
☐ Версия 2.0 возвращает все обязательные поля результата.

☐ Я заполнил журнал проблем и решений и указал ограничения.
☐ В данных и примерах нет персональной информации.
Чек-лист следует проходить по фактическому коду и выводу тестов, а не по наличию соответствующего TODO или описания. Для каждого отмеченного пункта должен существовать результат, который можно продемонстрировать.

12. Итог занятия и комплект сдачи

После выполнения работы Вы должны уметь объяснить, почему версия 2.0 надежнее первой и почему высокая точность на небольшом наборе не отменяет приемочное тестирование. Комплект сдачи должен позволять повторить проверку в новой сессии среды: сформировать данные, получить то же разделение, обучить модели и запустить приемочные примеры. Сохраненный только итоговый вывод без кода и параметров не подтверждает воспроизводимость результата.

- код формирования учебного датасета;
- неизмененная версия 1.0;
- функции prepare_ticket, calculate_priority и process_v2;
- обученная модель v2 и результаты сравнения accuracy;
- вывод приемочных тестов A01–A12;
- заполненный журнал проблем и решений;
- перечень ограничений и итоговый вывод.

Результат

Готова проверяемая версия 2.0 прототипа: она валидирует вход, классифицирует только достаточные данные, показывает уверенность, рассчитывает приоритет по понятным правилам и фиксирует случаи, где требуется специалист.
При этом прототип остается учебным решением: его готовность определяется не отсутствием любых ошибок, а наличием известных ограничений, контролируемого поведения и воспроизводимых проверок.